

Spring 2026 Final Cheat Sheet - Condensed

Built from: - Final Exam Sample.pdf - Final Exam Review - Spring 2026 (2).pdf

Use this version for printing. It is organized by problem type.

1. Static vs. Dynamic Binding

Main rule

- **virtual**: actual object type decides
- **not virtual**: declared pointer/reference/object type decides

Default arguments + virtual

Default arguments use the static type. The function body uses the dynamic type.

```
class A {
public:
    virtual void f(int x = 10) { cout << "A" << x; }
};
class B : public A {
public:
    void f(int x = 20) override { cout << "B" << x; }
};
A* p = new B();
p->f();    // B10
```

Key patterns

```
Base* p = new Derived();
Base& r = *p;
Base x = *p;
```

- `p->f()` with **virtual** -> `Derived::f`
- `r.f()` with **virtual** -> `Derived::f`
- `x.f()` -> `Base::f` because slicing happened

If `f` is not virtual: - `p->f()` -> `Base::f` - `r.f()` -> `Base::f` - `x.f()` -> `Base::f`

Scope resolution

```
obj.Base::f();
p->Base::f();
```

- forces the base version
- bypasses virtual dispatch

Slicing trigger

```
Base b = derivedObj;
Base b2 = *ptrToDerived;
```

- derived part is cut off
- `b` is now just a `Base`

Value vs. reference parameters

```
void f(Base b);    // slicing
void g(Base& b);   // polymorphism
```

2. Inheritance and Access

Direct access rules

Inside child member function:

Base member	Child can access?
public	Yes
protected	Yes
private	No

From `main`:

Member	main can access?
public	Yes
protected	No
private	No

Inheritance mode

Inheritance	Base public becomes	Base protected becomes
public	public	protected
protected	protected	protected
private	private	private

Base **private** never becomes directly visible in the child.

Common exam traps

- child can use base protected
- main cannot use protected
- with protected inheritance, even base public members stop being public to main

3. Constructors, Abstract Classes, and Destructors

Construction / destruction order

- construction: Base -> Derived
- destruction: Derived -> Base

If the derived constructor does not explicitly call base, base default constructor is used.

Virtual calls in constructors / destructors

Virtual dispatch does not reach the most-derived override inside constructors or destructors.

```
class A {
public:
    A() { f(); }
    virtual void f() { cout << "A"; }
};
class B : public A {
public:
    void f() override { cout << "B"; }
};
B b; // prints A
```

Abstract class

Pure virtual function:

```
virtual void print() const = 0;
```

Then: - base class is abstract - cannot create base objects - can use base pointers/references - derived must implement pure virtual functions or it stays abstract

Override rules

- pure virtual: derived must implement to become concrete
- regular virtual: derived may override, but does not have to

Deleting through base pointer

```
Base* p = new Derived();
delete p;
```

Safe only if base destructor is virtual.

If base destructor is not virtual: - only base destructor runs - derived cleanup skipped - derived memory may leak

4. Pointers, Stack vs. Heap, and Parameter Passing

Stack vs. heap

Thing	Where?
local variable	stack
local pointer variable	stack
local object	stack
object created by new	heap
member of stack object	stored inside that stack object

Example:

```
int* p = new int{7};
```

- p is on stack
- *p is on heap

Parameter passing

```
void f(int x)        // copy
void f(int& x)       // alias
void f(int* p)       // copied address
void f(int*& p)      // alias to pointer
```

Huge exam trap

```
void f(int* p) {
    p = new int{9};
}
```

This does not change the caller's pointer.

But:

```
void f(int* p) {
    *p = 9;
}
```

does change the caller's pointee.

Vectors

```
void f(vector<int>& a, vector<int> b)
```

- changes to `a` affect caller
- changes to `b` do not

5. Memory Leaks and Rule of 3

Leak checklist

For each `new` ask: - is there a matching `delete`? - is there a matching `delete[]` for `new[]`? - did the address get lost? - was the heap object itself never deleted?

Common leak patterns

```
int* p = new int{5}; // leak if never deleted
```

```
void f(int* p) {
    p = new int[4];
}
```

- local reassignment does not update caller
- new array leaks unless deleted inside `f`

Rule of 3

If a class owns dynamic memory, it usually needs: - destructor - copy constructor - copy assignment operator

If one is missing, default shallow copying can cause: - shared pointers by accident - double delete - memory leaks - unexpected output

Copy constructor vs assignment

```
Box a = b; // copy constructor
a = b; // copy assignment
```

These are not the same.

6. Overloading, Overriding, and Templates

Overloading

Compiler chooses based on argument types.

```
double func(double);
double func(int);
```

For nested calls, trace inside-out.

Overriding

Derived version replaces virtual base version during dynamic dispatch.

Exact signature rule

To override, the signature must match exactly.

```
class A {
public:
    virtual void f(int);
};
class B : public A {
public:
    void f(double); // NOT override
};
```

Function hiding

A derived function with the same name but different parameters hides base versions.

```
B b;
b.f(5); // calls B::f(double); A::f is hidden
```

Use `A::f(...)` or `using A::f;` to expose the base version.

override keyword

- optional
- does not change runtime behavior
- catches signature mismatches at compile time

Template substitution

Substitute the actual type first.

```
template <typename T>
double func(T x) { return x / 2; }
```

- `func(3) -> T = int -> integer division -> 1`
- `func(3.0) -> T = double -> 1.5`

Operator overloading

Treat overloaded operators like regular functions.

Ask: - what values are actually used? - what new object is returned? - what does `operator<<` print exactly?

7. stringstream, Exceptions, and Overflow

```
stringstream ss >> int
```

Stops at first invalid integer token.

```
stringstream ss("11 * 12 + 13");
while (ss >> x) { ... }
```

- reads 11
- fails at *
- loop ends

Exceptions

Once `throw` happens: - current function ends immediately - later statements in that function do not run

Overflow trap

This can overflow in the check itself:

```
if (x > INT_MAX - y)
```

Safer signed-addition check:

```
if ((y > 0 && x > INT_MAX - y) ||
    (y < 0 && x < INT_MIN - y)) {
    throw ...;
}
```

8. Recursion

Method

1. Find base case.
2. Find recursive step.
3. Write every call until base case.
4. Return upward.

Examples

```
int mystery(int x) {
    if (x > 8) return x;
    return mystery(x * 2);
}
```

Trace: - 1 -> 2 -> 4 -> 8 -> 16 - returns 16

Collatz-style:

```
if (n % 2 == 0) return 1 + foo(n/2);
if (n == 1) return 1;
return 1 + foo(3*n + 1);
```

Write every `n` value. Do not skip steps.

9. Linked List Patterns

Traversal

```
Node* curr = head;
while (curr != nullptr) {
    curr = curr->next;
}
```

Average

Plan: - if empty, throw - traverse once - track sum and count - return `sum / count`

Remove all maximums

Plan: 1. if empty, return 2. first pass: find max 3. remove matching nodes at head 4. remove matching nodes in rest using `prev` and `curr`

Reverse

```
Node* prev = nullptr;
Node* curr = head;
while (curr != nullptr) {
    Node* next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
head = prev;
```

Middle of list

Use slow/fast pointers:

```
Node* slow = head;
Node* fast = head;
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
return slow;
```

Empty list -> `nullptr`

Detect cycle

Floyd:

```
Node* slow = head;
Node* fast = head;
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) return true;
}
return false;
```

Swap evens between two lists

Safest exam method: - find next even node in list 1 - find next even node in list 2 - swap their values - continue until either list runs out of even nodes

```
Node* a = head;
Node* b = other.head;

while (true) {
    while (a != nullptr && a->value % 2 != 0) a = a->next;
    while (b != nullptr && b->value % 2 != 0) b = b->next;
    if (a == nullptr || b == nullptr) return;

    int temp = a->value;
    a->value = b->value;
    b->value = temp;

    a = a->next;
    b = b->next;
}
```

Why this is good: - no allocation - no pointer surgery - no head-update bugs

10. Struct / 2-D Vector Processing

Problem shape from sample exam:

- input: `vector<bounds>`
- output: `vector<vector<int>>`
- skip invalid bounds where `min > max`
- build inclusive rows from `min` to `max`
- trim all rows to shortest valid row length

Plan

1. create output vector
2. track shortest valid row
3. build each valid row
4. trim all rows to shortest length

Skeleton:

```
vector<vector<int>> out;
bool have_row = false;
int shortest = 0;

for (int i = 0; i < input.size(); i++) {
    if (input[i].min > input[i].max) continue;

    vector<int> row;
    for (int x = input[i].min; x <= input[i].max; x++) {
        row.push_back(x);
    }

    if (!have_row || row.size() < shortest) {
        shortest = row.size();
        have_row = true;
    }

    out.push_back(row);
}

for (int i = 0; i < out.size(); i++) {
    while (out[i].size() > shortest) {
        out[i].pop_back();
    }
}
```

If all bounds invalid, return empty vector.